nsconv 0.1.0-rc1 · Octubre 2026

Kit de prueba de nsconv

Versión de prueba 0.1.0-rc1 de punta a punta: NetStream simulado, FlowLink y un PCE de prueba, con un paquete de evidencias.

Herramienta comunitaria no oficial. nsconv no es un producto de Illumio y no tiene soporte de Illumio. Se entrega tal cual ("as is"), sin garantía de ningún tipo (licencia MIT).

Código y versiones: github.com/roschereric/netstream-converter

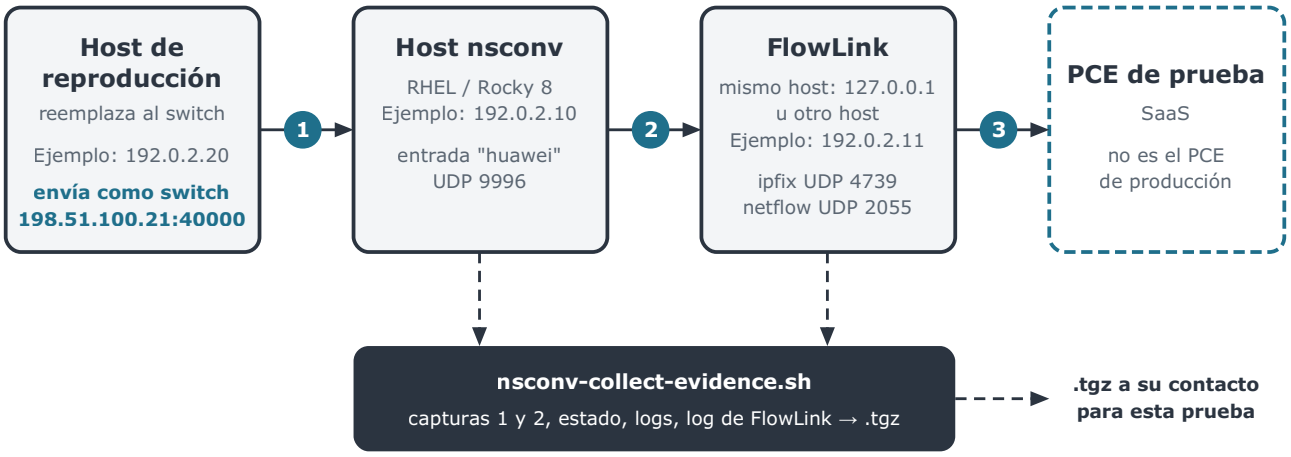
Contenido

Resumen 3
Instalar y configurar el laboratorio 5
Modo debug y dónde mirar 8
Simular NetStream 10
Qué probar 13
Reunir las evidencias 15
Cómo se validan las evidencias 16
Anexo: autoverificación del generador 17

Resumen

nsconv recibe Huawei NetStream (NetFlow v5, v9 e IPFIX con particularidades de Huawei) y lo reexporta como IPFIX, NetFlow v9 o NetFlow v5 limpio hacia Illumio FlowLink. Este kit prueba la versión de prueba **0.1.0-rc1** de punta a punta en un laboratorio: un host de reproducción envía NetStream grabado o generado, nsconv lo convierte, FlowLink envía los flujos a un **PCE de prueba** y usted devuelve un archivo de evidencias por host. No se necesita un switch Huawei ni el PCE de producción.

Laboratorio de prueba y flujos de datos (todos los nombres y direcciones son ejemplos)



#	Flujo	Contenido
1	Host de reproducción → nsconv, UDP 9996	NetStream v5, v9 o IPFIX, origen `198.51.100.21:40000` (el switch de ejemplo; 40000 es el puerto de origen de exportación por defecto de Huawei[1]).
2	nsconv → FlowLink, UDP 4739 (IPFIX) o 2055 (v9/v5)	Registros limpios. Con `source_address = "exporter"` (por defecto) cada datagrama conserva como origen la IP y el puerto del switch, así FlowLink ve a los switches mismos.
3	FlowLink → PCE de prueba, HTTPS	Resúmenes de flujos agregados, cada `aggregation_minutes` (por defecto 10, mínimo 5)[2].

Qué necesita

- **Host nsconv**: RHEL, Rocky o Alma Linux 8 x86_64 (9 también funciona), acceso root, `tcpdump`.
- **Host de reproducción**: cualquier VM Linux en el **mismo segmento de capa 2** que el host nsconv, con Python 3 y `tcpreplay` (EPEL 8 y 9 incluyen tcpreplay 4.6.1[3]).
- **FlowLink 2.0.1** en el host nsconv o en otro host del mismo segmento, asociado a un **PCE de prueba** (una API key de ese PCE).
- Los tres archivos del kit: esta guía, nsconv-sample-pcap.py (escribe capturas de prueba de NetStream al estilo Huawei con datos sintéticos) y nsconv-collect-evidence.sh (reúne las evidencias).

FUENTES

1. [official-Huawei, YANG] módulo huawei-netstream, source-ipv4-port por defecto 40000 — netconfcentral.org/modules/huawei-netstream/2019-06-27
2. [official-Illumio] Configure Flowlink — product-docs-repo.illumio.com/Tech-Docs/Core/26.1/Install-Upgrade-Admin/out/en/flowlink-2-0-0-configuration-and-usage/configure-flowlink.html
3. [official-Fedora] paquete tcpreplay (EPEL 8, EPEL 9) — packages.fedoraproject.org/pkgs/tcpreplay/tcpreplay/

Instalar y configurar el laboratorio

PASO 1 Instalar nsconv 0.1.0-rc1 en el host nsconv

Descargue el RPM y `SHA256SUMS` de la carpeta `releases/0.1.0-rc1/` de github.com/roschereric/netstream-converter, verifíquelos e instale. La versión del RPM es `0.1.0~rc1` (RPM no admite "-" en una versión; la versión final 0.1.0 la actualiza).

HOST NSCONV

```
# Descargar (o guardar ambos archivos de la carpeta con un navegador)
BASE=https://github.com/roschereric/netstream-converter/raw/main/releases/0.1.0-rc1
curl -LO $BASE/nsconv-0.1.0~rc1-1.x86_64.rpm -LO $BASE/SHA256SUMS
sha256sum -c --ignore-missing SHA256SUMS      # debe mostrar: nsconv-0.1.0~rc1-1.x86_64.rpm: OK
sudo dnf install ./nsconv-0.1.0~rc1-1.x86_64.rpm
nsconvctl version
```

El RPM instala `/usr/bin/nsconvctl`, la unidad `nsconv.service` (usuario `nsconv`, única capacidad `CAP_NET_RAW`), la configuración `/etc/nsconv/nsconv.toml` y un archivo sysctl que sube `net.core.rmem_max` a 8 MiB cuando es menor. El servicio todavía no arranca.

PASO 2 Configuración mínima de nsconv para el laboratorio

Los valores por defecto ya sirven para el laboratorio: entrada `huawei` en UDP 9996 y una salida IPFIX a `127.0.0.1:4739` enviada desde la dirección del switch (`source_address = "exporter"`). Solo hay que definir la lista de permitidos: por defecto está vacía, así que se descarta todo. `nsconvctl config set` conserva los comentarios, valida, escribe y recarga.

HOST NSCONV

```
# Permitir los switches de ejemplo (el host de reproducción envía con estas IP de origen)
sudo nsconvctl config set input.huawei.allow '["198.51.100.0/24"]'
# FlowLink en otro host: apuntar la salida hacia él (ejemplo 192.0.2.11)
sudo nsconvctl config set output.flowlink.target 192.0.2.11:4739
# Abrir el puerto de entrada al host de reproducción e iniciar el servicio
sudo firewall-cmd --permanent --add-port=9996/udp && sudo firewall-cmd --reload
sudo systemctl enable --now nsconv
sudo nsconvctl status
```

> **exporter o local**
>
> **exporter** (por defecto): FlowLink ve la IP de cada switch, por eso su `remote_addrs` lista las IP de los switches, igual que si los switches enviaran directo a FlowLink. FlowLink debe estar en el host nsconv o en el mismo segmento de capa 2. **local**: nsconv envía desde su propia dirección; `remote_addrs` es `127.0.0.1` (mismo host) o la IP del host nsconv. Se cambia con `sudo nsconvctl config set output.flowlink.source_address local`.

PASO 3 FlowLink contra el PCE de prueba

Instale FlowLink 2.0.1 según la guía de FlowLink, guarde la API key y el secreto del **PCE de prueba** en `/etc/illumio-flowlink/api_info`[1] y use un consumer por protocolo. FlowLink no tiene puertos por defecto y escucha en los puertos que usted configura[2]. Nunca reutilice un puerto de FlowLink para la entrada de nsconv.

/ETC/ILLUMIO-FLOWLINK/CONFIG.YAML (HOST FLOWLINK)

```
pce_addr: test-pce.example.com:443          # ejemplo: su PCE DE PRUEBA
org_id: 1                                   # ejemplo: org del PCE de prueba (SaaS)
api_key: $cat /etc/illumio-flowlink/api_info
data_directory: /var/lib/illumio-flowlink/data
aggregation_minutes: 5                      # mínimo: los flujos llegan antes al PCE
consumers:
  - name: nsconv-ipfix                      # protocolo de salida de nsconv = "ipfix"
    parser:
      type: ipfix
    connectors:
      - type: udp
        properties:
          ports: '4739'
          remote_addrs: ['198.51.100.21', '198.51.100.22']   # modo exporter: IP de los switches
  - name: nsconv-netflow                    # protocol = "netflow9" o "netflow5"
    parser:
      type: netflow
    connectors:
      - type: udp
        properties:
          ports: '2055'
          remote_addrs: ['198.51.100.21', '198.51.100.22']
```

HOST FLOWLINK

```
sudo systemctl restart illumio-flowlink
sudo -u ilo-flowlink /usr/local/bin/illumio-flowlink-ctl status
ss -lunp | grep flowlink                  # 4739 y 2055 escuchando
# Solo con FlowLink en otro host: abrir sus puertos al host nsconv
sudo firewall-cmd --permanent --add-port=4739/udp --add-port=2055/udp && sudo firewall-cmd --
reload
```

`remote_addrs` acepta IP individuales, no CIDR[2]. Con `source_address = "local"` use `'127.0.0.1'` (mismo host) o la IP del host nsconv (ejemplo `'192.0.2.10'`). En el PCE, las IP de los flujos aparecen en Illumination solo cuando pertenecen a workloads[3]: si las IP de ejemplo no aparecen, cree workloads no gestionados (unmanaged) para algunas de ellas en el PCE de prueba.

FUENTES

1. [official-Illumio] Deploy Flowlink as a Linux service — product-docs-repo.illumio.com/…/configure-flowlink/deploy-flowlink-as-a-linux-service.html
2. [official-Illumio] Ingested flow examples (consumers, ports, remote_addrs) — product-docs-repo.illumio.com/…/configure-flowlink/ingested-flow-examples.html

3. [official-Illumio] Scale and limitations — product-docs-repo.illumio.com/…/overview/scale-and-limitations.html

Modo debug y dónde mirar

nsconv

El nivel de log es la clave de configuración `log.level` (`error`, `warn`, `info`, `debug`); no hay una opción aparte. Al cambiarlo, el daemon recarga sin reiniciar. En `debug`, cada datagrama o flujo descartado, recortado o completado con un valor por defecto registra una línea por tipo de evento, como máximo 5 líneas cada 10 s por evento y exportador y 2000 líneas cada 10 s en total ("N similar suppressed").

HOST NSCONV

```
sudo nsconvctl config set log.level debug      # volver con: ... log.level info
nsconvctl logs -f                             # seguir el journal de nsconv
nsconvctl logs --since 30m                     # o "2026-10-21 14:00"
sudo nsconvctl logs --recent 200               # el buffer en memoria del daemon
journalctl -u nsconv NSCONV_EXPORTER=198.51.100.21:40000 -o verbose   # un switch
sudo nsconvctl status --detail                 # salud, alertas, exportadores, salidas, contadores
sudo nsconvctl status --exporter 198.51.100.21  # un switch: muestreo, desfase, pérdida de
secuencia
sudo nsconvctl --json status --detail
# lo mismo en JSON (accounting.unaccounted debe ser 0)
sudo nsconvctl tui --lang es                   # vista de pantalla completa, pestaña de alertas
```

Las advertencias tienen identificadores estables (`sampling-default`, `restamped`, `no-template`, `seq-gap`, `collector-down`, …). Se silencia una con `sudo nsconvctl dismiss <id>`; vuelve si su valor se duplica.

Colector de depuración y capturas en bruto

`nsconvctl decode --listen` es un colector pequeño que muestra cada flujo que recibe. Úselo en un puerto que FlowLink no use (ejemplo 9997).

HOST NSCONV

```
# a) Lo que envía el host de reproducción, antes de nsconv: reproducir al puerto 9997 en lugar de
9996
nsconvctl decode --listen 0.0.0.0:9997
# b) Lo que envía nsconv: agregar una segunda salida a /etc/nsconv/nsconv.toml y luego: sudo
nsconvctl reload
#    [[output]]
#    name = "debug"
#    protocol = "ipfix"
#    target = "127.0.0.1:9997"
#    source_address = "local"
nsconvctl decode --listen 127.0.0.1:9997 --count 50
# c) Datagramas NetStream en bruto de un switch a un pcap (usa tcpdump; por defecto 60 s / 2000
datagramas)
sudo nsconvctl debug-capture 198.51.100.21 --secs 120
# d) Decodificar cualquier captura sin conexión
nsconvctl decode nsconv-capture-198.51.100.21-*.pcap
```

FlowLink

FlowLink escribe sus logs en `/var/log/illumio-flowlink/`. Para una salida detallada, detenga el servicio e inícielo en modo verbose como `ilo-flowlink`[1]; la salida de debug muestra los registros entrantes, la IP, el puerto y el protocolo de cada registro, cada agregación y envío, y el código de respuesta del PCE[2]. Úselo solo durante la prueba y luego vuelva al servicio.

HOST FLOWLINK

```
sudo -u ilo-flowlink /usr/local/bin/illumio-flowlink-ctl stop
sudo -u ilo-flowlink /usr/local/bin/illumio-flowlink-ctl start -v
sudo tail -f /var/log/illumio-flowlink/flowlink.log
# después de la prueba
sudo -u ilo-flowlink /usr/local/bin/illumio-flowlink-ctl stop && sudo systemctl start illumio-flowlink
```

FUENTES

1. [official-Illumio] Deploy Flowlink as a Linux service (directorio de logs, start -v) — product-docs-repo.illumio.com/…/deploy-flowlink-as-a-linux-service.html
2. [official-Illumio] Flowlink troubleshooting (salida de debug) — product-docs-repo.illumio.com/…/flowlink-usage/troubleshooting.html
3. [official-systemd] journalctl(1), filtros por campo — freedesktop.org/software/systemd/man/latest/journalctl.html

Simular NetStream

Generar una captura de prueba

`nsconv-sample-pcap.py` escribe un pcap de datagramas NetStream al estilo Huawei: NetFlow v5 (registros de 48 bytes), v9 (formato de plantilla de flujos originales de Huawei, con el campo privado 232 de Huawei) o IPFIX (tiempos relativos más el IE 160 en cada registro, campos enterprise de Huawei), puerto UDP de origen 40000, plantillas en datagramas propios antes de los datos y repetidas cada 10 datagramas de datos. Solo necesita Python 3.6 o posterior (RHEL 8: `/usr/libexec/platform-python`). Todos los valores son sintéticos.

HOST DE REPRODUCCIÓN

```
curl -LO https://roschereric.github.io/netstream-converter/test-kit/nsconv-sample-pcap.py
# MAC de la NIC del host nsconv (ejecutar en el host nsconv): ip -br link show <nic>
python3 nsconv-sample-pcap.py --version 9 --switch 198.51.100.21 --dst 192.0.2.10 \
    --dst-mac 52:54:00:12:34:56 --flows 200 --out v9.pcap --expect v9.expect.jsonl
```

Opción	Significado (por defecto)
`--version 5|9|ipfix`	Versión de exportación (9).
`--switch IP[,IP…]`	Dirección(es) de origen del switch; cada switch recibe sus propios flujos (198.51.100.21).
`--sport` / `--dst` / `--dport`	Puerto de origen del switch (40000), IP del host nsconv (192.0.2.10), puerto de entrada de nsconv (9996).
`--dst-mac` / `--src-mac`	MAC de las tramas. Para tcpreplay, `--dst-mac` debe ser la NIC de nsconv.
`--src` / `--dst-hosts` / `--dst-ports`	IP o CIDR de workloads y puertos (10.1.1.0/24 → 10.2.2.0/24, puertos 443,22,3306,8080,53; IPv6: 2001:db8:a::/112 → 2001:db8:b::/112).
`--flows N` / `--ipv6`	Flujos por switch (200) / workloads IPv6 (solo v9 e IPFIX).
`--sampling N`	Tasa 1 de N escrita como IE 34 en cada registro v9/IPFIX (0 = no informada, como hacen muchos switches Huawei).
`--domain N`	Source ID v9 / Observation Domain IPFIX / engine id v5 (0): un segundo valor emula una segunda tarjeta de línea.
`--pps` / `--template-every` / `--data-only`	Tasa de datagramas en los tiempos del pcap (100); repetición de plantilla (10); datos sin plantillas (prueba de reinicio).
`--expect FILE` / `--check`	Escribir los flujos generados como líneas JSON / decodificar el pcap con `nsconvctl` y comparar cada flujo.

Genere justo antes de reproducir

Los tiempos de los flujos se fijan al generar (terminan entre 1 y 30 s antes). Un pcap reproducido más de 5 minutos después queda fuera de `convert.max_clock_skew_secs = 300`: nsconv desplaza entonces los tiempos al momento de recepción y cuenta `restamped`. Mantenga sincronizados los relojes de todos los hosts (chrony).

Opción A: `nsconvctl replay` (simple)

Envía el contenido NetStream de una captura con un socket UDP normal. Nunca falsifica el origen: nsconv y FlowLink ven la dirección del **host de reproducción** (ejemplo 192.0.2.20) y un puerto de origen efímero. Sirve para una primera prueba; para las pruebas de identidad del switch use la opción B.

HOST DE REPRODUCCIÓN Y HOST NSCONV

```
# host nsconv: permitir el host de reproducción; remote_addrs de FlowLink lista entonces
192.0.2.20 (modo exporter)
sudo nsconvctl config set input.huawei.allow '["198.51.100.0/24", "192.0.2.20"]'
# host de reproducción: copiar nsconvctl del RPM o el binario de la versión, luego
nsconvctl replay v9.pcap --to 192.0.2.10:9996 --rate 200
```

Opción B: tcpreplay (conserva la dirección de origen del switch)

tcpreplay envía las tramas tal como están en el pcap, así los datagramas llegan desde `198.51.100.21:40000`: la lista de permitidos, la vista por switch y `source_address = "exporter"` se comportan como con un switch real. El generador ya escribe el destino correcto; cualquier otra captura (por ejemplo una de `debug-capture`, que es una captura Linux “cooked”) se reescribe antes con tcprewrite.

HOST DE REPRODUCCIÓN

```
sudo dnf install epel-release && sudo dnf install tcpreplay   # Rocky/Alma; RHEL: habilitar EPEL
antes
# Solo para capturas que no vienen del generador: IP del colector anterior (ejemplo 192.0.2.50)
hacia el
# host nsconv, su MAC, checksums recalculados. Agregue --dlt=enet para una captura cooked (debug-
capture).
tcprewrite --infile=capture.pcap --outfile=lab.pcap \
    --enet-dmac=52:54:00:12:34:56 --enet-smac=02:00:00:00:00:01 \
    --dstipmap=192.0.2.50/32:192.0.2.10/32 --fixcsum
# Reproducir por la NIC que da al host nsconv
sudo tcpreplay --intf1=eth0 v9.pcap                          # tiempos del pcap (100 datagramas/s)
sudo tcpreplay --intf1=eth0 --pps=2000 --loop=30 burst.pcap   # prueba de carga
```

Opción de tcpreplay / tcprewrite	Efecto (manual oficial)
`--intf1` (`-i`)	Interfaz de salida; obligatoria.
`--pps` / `--mbps` / `--topspeed`	Paquetes por segundo / Mbps / lo más rápido posible; por defecto, los tiempos del pcap.
`--loop` / `--duration` / `--limit`	Repetir el archivo N veces / parar tras N segundos / tras N paquetes.
`--dstipmap` (`-D`)	Pseudo-NAT de las IP de destino, pares `CIDR:CIDR`.
`--enet-dmac` / `--enet-smac`	Reemplazar la MAC de destino / de origen.
`--dlt=enet`	Convertir una captura Linux cooked a Ethernet (requiere ambas MAC).
`--fixcsum` (`-C`)	Recalcular los checksums IPv4/TCP/UDP.

No use `--unique-ip`: cambia la cabecera IP (la dirección del switch), no las direcciones dentro de los registros NetStream.

> **Reproduzca desde otro host, nunca desde el host nsconv**
>
> tcpreplay escribe las tramas directamente en el lado de transmisión de la NIC. La pila de red del host nunca recibe lo que él mismo transmite, y el switch no devuelve una trama por el mismo puerto por el que llegó, así que una reproducción en el propio host nsconv nunca llega a nsconv. Use una segunda VM o host en el mismo segmento de capa 2 (misma VLAN, sin router en medio).

Verificaciones de red en el host nsconv

- **Filtro de ruta inversa**: el valor por defecto de RHEL `rp_filter = 1` (estricto) descarta un datagrama cuyo origen (198.51.100.21) se enrutaría de vuelta por otra interfaz. La ruta por defecto por la NIC receptora pasa. Si el host nsconv tiene varias NIC, agregue una ruta para el rango de los switches por la NIC receptora o use el modo flexible en ella: `sudo sysctl -w net.ipv4.conf.<nic>.rp_filter=2`. Los descartes aparecen en `nstat -az TcpExtIPReversePathFilter`, no en nsconv.
- **firewalld**: UDP 9996 abierto en la zona de la NIC receptora (paso 2).
- **Lista de permitidos**: `input.huawei.allow` contiene el rango de los switches (opción B) o la IP del host de reproducción (opción A); el resto se cuenta como `denied`.
- **Verificar la llegada**: `sudo tcpdump -ni any udp port 9996 -c 5` y luego `sudo nsconvctl status`.

FUENTES

1. [official-tcpreplay] manual de tcpreplay(1) — tcpreplay.appneta.com/reference/man/tcpreplay/
2. [official-tcpreplay] manual de tcprewrite(1) — tcpreplay.appneta.com/reference/man/tcprewrite/
3. [official-tcpreplay] descripción de tcprewrite (conversión DLT) — tcpreplay.appneta.com/tools/tcprewrite/
4. [official-kernel] ip-sysctl: rp_filter 0/1/2 — docs.kernel.org/networking/ip-sysctl.html
5. [lab] VM de prueba Rocky 8.10, 2026-10-20: rp_filter descartó las tramas con un origen fuera de las rutas del enlace (contador TcpExtIPReversePathFilter), y llegaron una vez que existió una ruta de vuelta por la interfaz receptora.

Qué probar

Ejecute el script de evidencias (siguiente sección) alrededor de cada prueba o grupo de pruebas y escriba los números de prueba en `--note`. “Estado” significa `sudo nsconvctl status --detail`; “Explorer” significa el Explorer del PCE de prueba, filtrado por las IP de workloads de ejemplo. Los flujos llegan al PCE después del intervalo de agregación de FlowLink.

#	Prueba	Cómo	Resultado esperado	Dónde
1	Instalación y sandbox	Pasos 1–2; `systemd-analyze security nsconv`	Servicio activo; exposición “OK”; proceso con usuario `nsconv` y solo CAP_NET_RAW	Evidencia: nsconv-security, nsconv-process
2	v9 IPv4 → IPFIX (base)	Generar `--version 9 --flows 200`, tcpreplay	`flows_in` = `flows_out` = 200, `unaccounted` 0; FlowLink acepta 198.51.100.21; flujos en Explorer	Estado, log de FlowLink, Explorer
3	Entrada v5	`--version 5`	El exportador muestra v5; tasa “default 1000” y alerta `sampling-default` (la cabecera v5 no trae tasa, como en las capturas Huawei conocidas)	Estado, Explorer
4	Entrada IPFIX	`--version ipfix`	Mismos flujos; campos enterprise de Huawei ignorados: `malformed`, `template_rejects` 0	Estado, Explorer
5	Salida v9 y v5	`config set output.flowlink.protocol netflow9` y `target …:2055`; luego `netflow5`	Flujos en Explorer por el consumer netflow; la salida v5 cuenta los flujos IPv6 como `v6_on_v5` y no los envía	Estado, log de FlowLink, Explorer
6	Identidad del switch (exporter)	`--switch 198.51.100.21,198.51.100.22`; luego quitar .22 de `remote_addrs`	FlowLink acepta y lista ambas IP; después de quitarla, FlowLink descarta solo .22	Log de FlowLink, Explorer
7	Modo local	`source_address local`, `remote_addrs = 127.0.0.1` o la IP de nsconv	Flujos aceptados; anote cómo nombra el log de FlowLink a un exportador 127.0.0.1	Log de FlowLink
8	Dos tarjetas de línea, una IP	Dos pcap, mismo `--switch`, `--domain 1` / `--domain 2`, distinto `--src`	Dos sesiones en nsconv; ambos conjuntos de workloads en Explorer (FlowLink separa los dominios)	Estado, Explorer
9	Muestreo (IE 34)	`--sampling 2048`; luego `config set input.huawei.sampling_override '{ "198.51.100.21" = 1000 }'`	Tasa 2048 “reported”, luego 1000 “override”, presente en cada registro de salida; anote si el PCE escala los contadores	Estado, pcap de salida, Explorer
10	Bytes (IE 1 y 85)	`--flows 1 --expect one.jsonl`	Explorer muestra el número de bytes de `one.jsonl`	Explorer

11	IPv6	`--ipv6` con v9 y con IPFIX	Flujos IPv6 decodificados y reenviados; anote si aparecen en Explorer	Estado, Explorer
12	Intervalos de tiempo	Generar y reproducir de inmediato; anotar la hora UTC	Flujos en el intervalo de la hora de reproducción; `restamped` 0	Explorer, estado
13	Desfase de reloj	Reproducir un pcap generado hace más de 10 min	`restamped` crece, alerta `restamped`; flujos fechados al momento de recepción	Estado, Explorer
14	Recarga	`config set log.level debug`, luego `info`	Mismo PID, “reload” en el log, los flujos continúan	Logs, estado
15	Reinicio, plantillas conservadas	Reproducir v9; `systemctl restart nsconv`; reproducir un pcap `--data-only` del mismo switch	Flujos de solo datos decodificados (`no_template` 0): las plantillas se restauraron de `/var/lib/nsconv`	Estado
16	Colector caído	Mismo host: `systemctl stop illumio-flowlink` durante una reproducción y luego iniciarlo	nsconv sigue activo; alerta `collector-down` en unos 15 s; los flujos vuelven tras el siguiente refresco de plantilla (20 s)	Estado, logs
17	Ráfaga	`--flows 50000 --pps 5000`, `tcpreplay --pps=5000 --loop=10`	Sin descartes del kernel (`RcvbufErrors` sin cambios), `unaccounted` 0, `flows_out` = `flows_in`	Evidencia: udp-snmp, estado
18	Socket de FlowLink	`ss -lunp | grep flowlink`	Anote si escucha en IPv4, IPv6 o dual-stack; la evidencia copia el schema de configuración de FlowLink	Evidencia: ss-udp
19	SELinux, firewall	Toda la prueba	Ninguna denegación AVC para nsconv ni FlowLink	Evidencia: avc-window, selinux

> **Respuestas que buscamos**
>
> Las pruebas 6–11 y 18 muestran cómo trata FlowLink 2.0.1 los orígenes con IP de switch desde un relay en el mismo host, los dominios de tarjetas de línea, la tasa de muestreo, los contadores de bytes (IE 1 u 85), IPv6 y los intervalos de tiempo. Escriba lo que ve en el PCE junto al número de prueba en `--note`, o en su mensaje junto con el archivo.

FUENTES

1. [nsconv] contadores, alertas y comandos documentados en la versión (nsconvctl --help, guía de operación) — github.com/roschereric/netstream-converter
2. [official-Illumio] Flowlink scale and limitations — product-docs-repo.illumio.com/…/overview/scale-and-limitations.html

Reunir las evidencias

`nsconv-collect-evidence.sh` se ejecuta como root durante una ventana de tiempo (por defecto 300 s). Captura con tcpdump la entrada NetStream (switch → nsconv) y cada salida de nsconv (nsconv → colector), cada archivo con un tope de 100 MB, y reúne todo lo necesario para verificar la prueba. No cambia nada en el host; una herramienta o un archivo que falte queda registrado y la ejecución sigue.

HOST NSCONV

```
curl -LO https://roschereric.github.io/netstream-converter/test-kit/nsconv-collect-evidence.sh
sudo bash nsconv-collect-evidence.sh --dry-run            # muestra lo que va a ejecutar
sudo bash nsconv-collect-evidence.sh --secs 300 --sample v9.pcap --note "pruebas 2, 6: v9, dos
switches"
# ahora inicie la reproducción en el host de reproducción; debe terminar dentro de la ventana
# FlowLink en otro host: ejecutar allí al mismo tiempo
sudo bash nsconv-collect-evidence.sh --flowlink-only --secs 300
```

En el .tgz	Contenido
`pcap/`	`input.pcap`, `output-<nombre>.pcap` (host FlowLink: `flowlink-input.pcap`), cada uno decodificado por `nsconvctl decode` en `.flows.jsonl`
`cmd/`	`nsconvctl version`, `status --json --detail` antes y después, `logs --since` journal (JSON), `config show`, estado de la unidad, `systemd-analyze security`, sysctl de rp_filter y buffers, firewalld, nftables, SELinux y denegaciones AVC, `ss -lunp`, `ip addr/route`, reloj (chrony), versión, servicio e `illumio-flowlink-ctl status` de FlowLink
`files/`	`nsconv.toml`, `config.yaml` de FlowLink con los secretos reemplazados por `<REDACTED>`, las líneas del log de FlowLink de la ventana, el schema de configuración de FlowLink, sus pcap de muestra
`MANIFEST.tsv`, `SHA256SUMS`, `SUMMARY.txt`, `NOTE.txt`	Cada elemento con su código de salida y su SHA-256; sumas para `sha256sum -c`; flujos por captura y `unaccounted`; su nota

Envíelo

El script muestra el nombre del archivo `nsconv-evidence-<host>-<hora UTC>.tgz` y su SHA-256. Envíe el .tgz (y el del host FlowLink, si es otro) con el SHA-256 y sus notas por su canal habitual a su contacto para esta prueba. El archivo de API key de FlowLink nunca se lee, y las API keys, secretos, tokens y contraseñas de `config.yaml` se reemplazan; las capturas contienen solo los flujos sintéticos de prueba.

FUENTES

1. [official-tcpdump] tcpdump(1): -c, -U, -w, -i any — tcpdump.org/manpages/tcpdump.1.html
2. [lab] VM de prueba Rocky 8.10, 2026-10-20: el script se ejecutó como root (nsconv en modo exporter, colector en 127.0.0.1) y en modo `--flowlink-only` con una configuración de FlowLink de muestra; los secretos se ocultaron y cada elemento quedó en MANIFEST.tsv.

Cómo se validan las evidencias

Una prueba es correcta cuando todos los puntos siguientes se cumplen en su ventana. Las verificaciones usan solo lo que contiene el .tgz.

- ☐ **Entrada = generado** — los flujos de `pcap/input.flows.jsonl` son iguales a los flujos generados (archivo `--expect`, o el pcap de muestra decodificado otra vez).
- ☐ **Salida = entrada** — `pcap/output-<nombre>.flows.jsonl` contiene los mismos flujos (direcciones, puertos, protocolo, paquetes, bytes, tiempos), cada uno exactamente una vez, con la IP y el puerto del switch como exportador en modo exporter.
- ☐ **Nada sin contabilizar** — en `nsconv-status-after.json`, `accounting.unaccounted` es 0, la diferencia de `flows_in` y `flows_out` entre antes y después es igual a los flujos reproducidos, `denied`, `malformed` y `send_errors` no crecieron, y `RcvbufErrors` en `udp-snmp` no cambió.
- ☐ **FlowLink aceptó los switches** — `files/flowlink-window.log` nombra las IP de los switches (o 127.0.0.1 / el host nsconv en modo local) y no muestra descartes ni “bad data” para ellas.
- ☐ **El PCE los recibió** — el mismo log muestra los envíos al PCE de prueba con éxito (con el código de respuesta del PCE en modo verbose), y Explorer muestra los flujos de workloads de la prueba (su nota).
- ☐ **Host limpio** — ninguna denegación AVC, servicio activo, ningún reinicio durante la ventana (`NRestarts`).

FUENTES

1. [nsconv] schema de estado `nsconv.status/1`: ecuaciones de contabilidad y residuo `unaccounted`, mostrados por `nsconvctl status`.

Anexo: autoverificación del generador

`--check` decodifica el pcap generado con `nsconvctl decode --json` y compara cada flujo (exportador, direcciones, puertos, protocolo, paquetes, bytes, tiempos de inicio y fin, tasa de muestreo) con lo generado. Ejecútelo en el host nsconv antes de una prueba.

HOST NSCONV

```
/usr/libexec/platform-python nsconv-sample-pcap.py --version ipfix --ipv6 --sampling 2048 --out
t.pcap --check
# check OK: nsconvctl decoded 200 of 200 flows with identical addresses, ports, protocol, packets,
bytes and times
```

Antes de esta versión se generaron y decodificaron sin diferencias diez variantes (v5; v5 con un segundo engine id; v9 e IPFIX, cada una IPv4 e IPv6, con y sin tasa de muestreo informada; dos switches cada una, 666 flujos por archivo), en macOS y en Rocky Linux 8.10 con `platform-python` 3.6. En Rocky 8.10, las tramas también se reprodujeron con la dirección de origen del switch hacia nsconv (modo exporter): las capturas de entrada y salida del script de evidencias contenían los mismos 500 flujos, `unaccounted` fue 0 y, después de un reinicio, una captura de solo datos se decodificó con las plantillas restauradas.

FUENTES

1. [lab] macOS y Rocky Linux 8.10 (platform-python 3.6.8), 2026-10-20: 10 de 10 variantes del generador decodificadas sin diferencias.

Herramienta comunitaria no oficial. nsconv no es un producto de Illumio y no tiene soporte de Illumio. Se entrega tal cual ("as is"), sin garantía de ningún tipo (licencia MIT).

Illumio, FlowLink y PCE son nombres de productos de Illumio, Inc.; Huawei y NetStream pertenecen a Huawei Technologies Co., Ltd. Se usan aquí solo para nombrar los productos con los que funciona nsconv. Valide los detalles técnicos con la documentación oficial de sus versiones de producto.

Código y versiones: github.com/roschereric/netstream-converter